Interactive Typing Toolkit: The Types You Want

Oliver Tušla
Czech Technical University in Prague
Prague, Czech Republic
tuslaoli@fit.cvut.cz

Mickaël Laurent
Charles University
Prague, Czech Republic
mickael.laurent@matfyz.cuni.cz

Pierre Donat-Bouillud
Czech Technical University in Prague
Prague, Czech Republic
pierre.donat.bouillud@fit.cvut.cz

Abstract

Set-theoretic type systems infer precise polymorphic types for unannotated functions in dynamic languages, capturing overloaded behaviour through intersections, unions, and negations of types. The price of this precision is readability: inferred signatures are often far more verbose than the type a programmer would write by hand. Worse, an overly precise signature propagates through inference into every definition that uses it, so simplifying one signature can yield more straightforward types across the entire codebase. We present an interactive typing toolkit — a VS Code extension backed by an LSP server on top of the MLsem typing library — that lets the programmer reshape an inferred type through simple, semantically justified operations such as *instantiation* of type variables and *merging* of overloads, exploring the spectrum of correct types for a function and choosing where on the precision–readability scale to stop. Readability is not the only concern: a narrower contract than the code happens to admit can also capture intent. We outline the toolkit's design and our plans to apply it to the R language.

Keywords: set-theoretic types, semantic subtyping, type inference, dynamic languages, IDE tooling

ACM Reference Format:
Oliver Tušla, Mickaël Laurent, and Pierre Donat-Bouillud. 2026. Interactive Typing Toolkit: The Types You Want. In *ECOOP-Companion (ECOOP-C '26), June 29–30, 2026, Brussels, Belgium.* ACM, New York, NY, USA, 2 pages. https://doi.org/10.1145/3849733.3849786

1 The type you get vs. the type you want

Dynamic languages are full of functions whose behaviour is richer than any simple type suggests: JavaScript's `||` operator, for one, is *not* boolean. `0 || "x"` evaluates to `"x"`, and `{} || 1` to `{}`. The operator is exactly this function [1]:

```
function logical_or(x, y) {
  if (x) { return x; } else { return y; }
}
```

The naive type $(\texttt{bool}, \texttt{bool}) \to \texttt{bool}$ rejects both uses above. A type system based on set-theoretic types instead infers a type accounting for all of them:

$$\forall \alpha, \beta.\ ((\alpha \wedge \texttt{truthy}, \texttt{any}) \to \alpha \wedge \texttt{truthy}) \wedge ((\texttt{falsy}, \beta) \to \beta)$$

This type is precise — but probably not the type you expected.

We present a VS Code extension that lets the programmer *interactively reshape* an inferred set-theoretic type, choosing for each function where on the precision–readability scale to stop.

2 Set-theoretic types

Our toolkit is built on the MLsem typing library [2], which implements polymorphic type inference with set-theoretic types and semantic subtyping. The guiding intuition is that a type *is* the set of values it accepts — its interpretation $[\![t]\!]$ — and each type connective is the matching set operation:

$$[\![\texttt{42}]\!] = \{\texttt{42}\} \qquad [\![\texttt{int}]\!] = \{\ldots, 0, 1, \ldots\}$$

$$[\![t_1 \wedge t_2]\!] = [\![t_1]\!] \cap [\![t_2]\!] \qquad [\![t_1 \vee t_2]\!] = [\![t_1]\!] \cup [\![t_2]\!]$$

Subtyping is then set inclusion: $s \le t$ exactly when $[\![s]\!] \subseteq [\![t]\!]$. The system provides the full algebra of sets — union, intersection, negation — plus singleton types such as `true` or `42`, a top type `any`, and a bottom type `empty`:

$$\texttt{true} \vee \texttt{false} = \texttt{bool} \qquad \texttt{int} \wedge \texttt{string} = \texttt{empty}$$

$$\neg\texttt{true} = \texttt{any} \setminus \texttt{true}$$

Functions are typed with arrows $t \to s$, and an *intersection of arrows* expresses overloading (ad hoc polymorphism): the type $(\texttt{int} \to \texttt{bool}) \wedge (\texttt{bool} \to \texttt{int})$ honours both contracts at once, and is strictly more precise than $\texttt{int} \vee \texttt{bool} \to \texttt{int} \vee \texttt{bool}$. Type variables α provide genericity (parametric polymorphism). Because subtyping is decided *semantically* rather than syntactically, inference gets remarkably precise — and hence verbose.

3 Refining a signature, interactively

Consider `logical_or` again, this time in MLsem. From the unannotated definition, MLsem infers an intersection of two overloads:

```
(* vals are inferred overloads *)
val logical_or : 'a & truthy -> 'b -> 'a & truthy
val logical_or : 'c & falsy -> 'd -> 'd
(* let is a definition from the user *)
let logical_or x y =
  if x is truthy then x else y
```

This work is licensed under a Creative Commons Attribution 4.0 International License.
ECOOP-C '26, Brussels, Belgium
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2540-1/2026/06
https://doi.org/10.1145/3849733.3849786

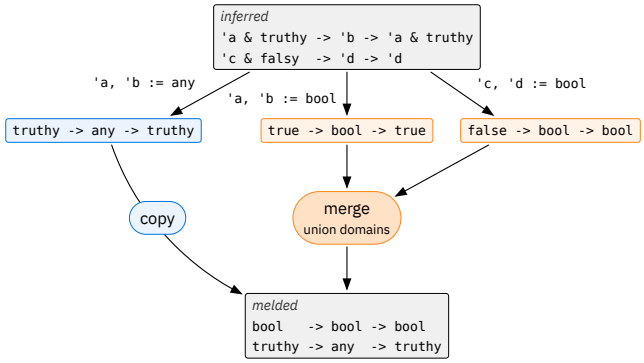


Figure 1. Instantiate and merge operations reshaping the inferred signature of `logical_or`.

Our toolkit reshapes such signatures with two operations:

Instantiate pin a type variable to a concrete type, per overload.

Merge collapse selected overloads into a single arrow by unioning their domains and codomains:

$$(t_1 \to s_1) \wedge (t_2 \to s_2) \quad \longmapsto \quad (t_1 \vee t_2) \to (s_1 \vee s_2).$$

The result is a supertype of the original intersection, losing precision in a controlled, predictable way.

From the inferred type above, applying the operations yields simpler and less precise signatures:

```
val logical_or : bool -> bool -> bool
val logical_or : truthy -> any -> truthy
```

The first is the familiar boolean reading and the second keeps the "returns its first argument when truthy" behaviour but drops the polymorphism. The toolkit makes the choice between them explicit.

The toolkit provides these operations using an integrated interface as a VS Code extension, as depicted in Figure 2. It is available at https://github.com/asmarcz/MLsem-vscode.

4 Future direction: moving to R

More operations are planned, notably *domain decomposition* and *branch specialization*.

Our target is the R language. It is deeply dynamic — coercions, promises, and multiple dispatch mechanisms — so precise inference produces unreadable types, while readable types require judgement calls that only the programmer can make. The toolkit eases this trade-off: inference supplies the precise starting point, the user steers the simplification.

Several directions remain open: LLM-assisted trade-off search guided by user-provided success criteria, operations for language-specific features such as builtin dispatch, and extending the approach to modules and objects. Runtime feedback could also guide the shape of the types.

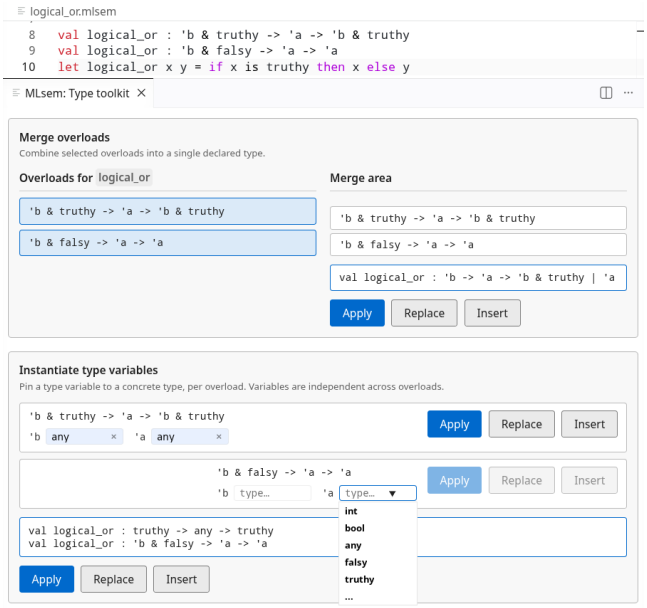


Figure 2. The merge and instantiate panels of the VS Code extension.

Acknowledgments

This work is supported by the Czech Ministry of Education, Youth and Sports under program ERC-CZ, grant agreement LL2325, as well as by the Czech Science Foundation Grant No. 23-07580X.

References

[1] Giuseppe Castagna, Mickaël Laurent, and Kim Nguyễn. 2024. Polymorphic Type Inference for Dynamic Languages. *Proc. ACM Program. Lang.* 8, POPL, Article 40 (Jan. 2024). doi:10.1145/3632882

[2] Mickaël Laurent and Jan Vitek. 2026. Type Inference for Functional and Imperative Dynamic Languages. *Proc. ACM Program. Lang.* 10, OOPSLA1, Article 112 (April 2026). doi:10.1145/3798220